

TCP HTCP_Modified Metrics Analysis

Modified Functions and Their Impacts:

1. CongestionAvoidance Function

- **Change:**
 - Introduced a less aggressive CWND growth mechanism by using the square root of `m_alpha` in the calculation of `adderr`.
 - Set a minimum increase of 2 segments for CWND growth, regardless of network conditions.
- **Potential Impact:**
 - **Slower CWND Growth:** The congestion window will grow more gradually, leading to a more cautious utilization of network bandwidth.
 - **Improved Stability:** This reduced aggressiveness minimizes the risk of buffer overflow, queuing delays, and packet loss in congested networks.
 - **Potential Throughput Reduction:** In networks with high bandwidth and low latency, this conservative growth may underutilize available capacity.

2. UpdateAlpha Function

- **Change:**
 - Set a higher initial value for `m_alpha` (2.0) to promote faster growth in the early stages of congestion avoidance.
 - Adjusted the alpha growth formula to use a cubic root function (`15 * pow(diffSec, 1.5)`) for less steeper increases as the elapsed time (`diffSec`) grows.
 - Ensured that `m_alpha` does not fall below 2.0.

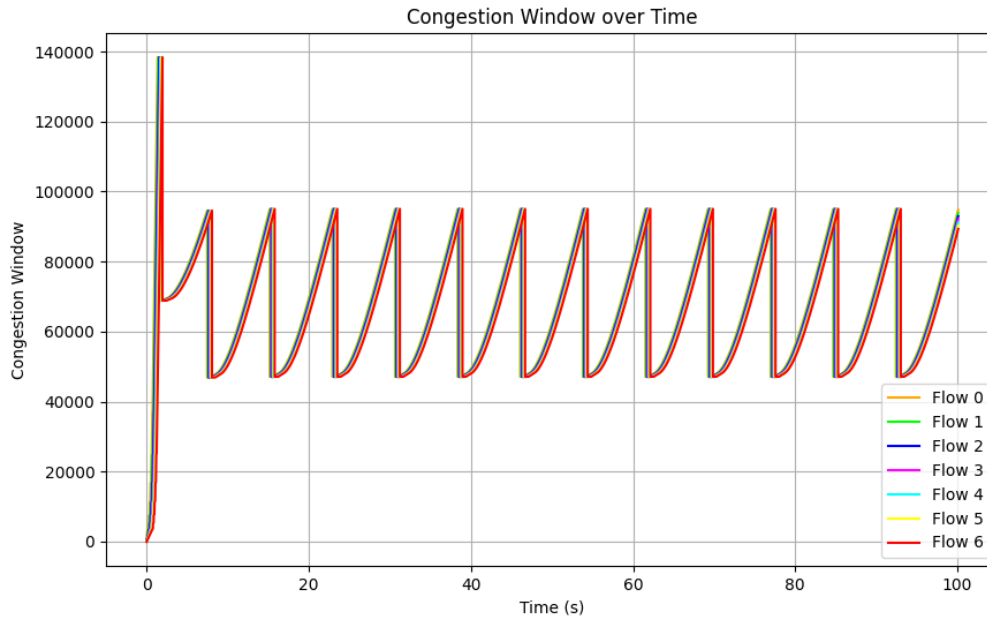
- **Potential Impact:**
 - **Less Aggressive Scaling:** The growth of alpha is slower compared to the original formula due to the use of the 1.5 power. This results in more gradual CWND growth over time, making the congestion control mechanism less aggressive.
 - **Better Stability:** The more gradual adjustment can enhance stability in fluctuating networks by preventing overly rapid increases in CWND, but it could also result in suboptimal performance in high-capacity networks.
 - **Moderate Adaptation to Bandwidth:** The slower scaling leads to more cautious adjustments, which might prevent over-utilization of network resources and reduce the risk of congestion and buffer overflow.

3. UpdateBeta Function

- **Change:**
 - Made `m_beta` dynamic by basing it on the ratio of minimum to maximum RTT (`m_minRtt / m_maxRtt`).
 - Introduced thresholds to adjust `m_beta`:
 - 0.8 if RTT ratio > 0.8 (good network conditions).
 - 0.7 if RTT ratio > 0.6 (moderate conditions).
 - 0.5 if RTT ratio ≤ 0.6 (poor conditions).
- **Potential Impact:**
 - **Adaptive Backoff:** The dynamic adjustment of `m_beta` allows the algorithm to reduce the CWND less aggressively in favorable conditions (e.g., low RTT) and more conservatively in unfavorable ones.
 - **Improved Stability:** This reduces unnecessary throughput loss during minor congestion events and enhances fairness among competing flows.
 - **Reduced Congestion Reaction:** In highly congested networks, the less aggressive backoff (when `m_beta` is high) may exacerbate congestion and increase retransmissions.

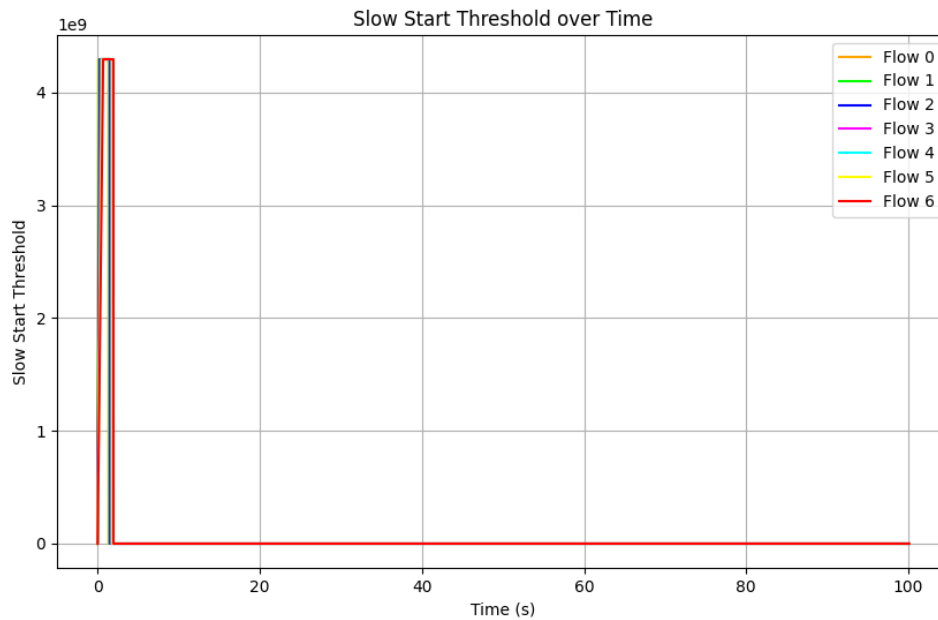
Graphs

Congestion Window (cwnd): A TCP state variable that controls the amount of data a TCP sender can send into the network before receiving an acknowledgment (ACK) for previously sent data.



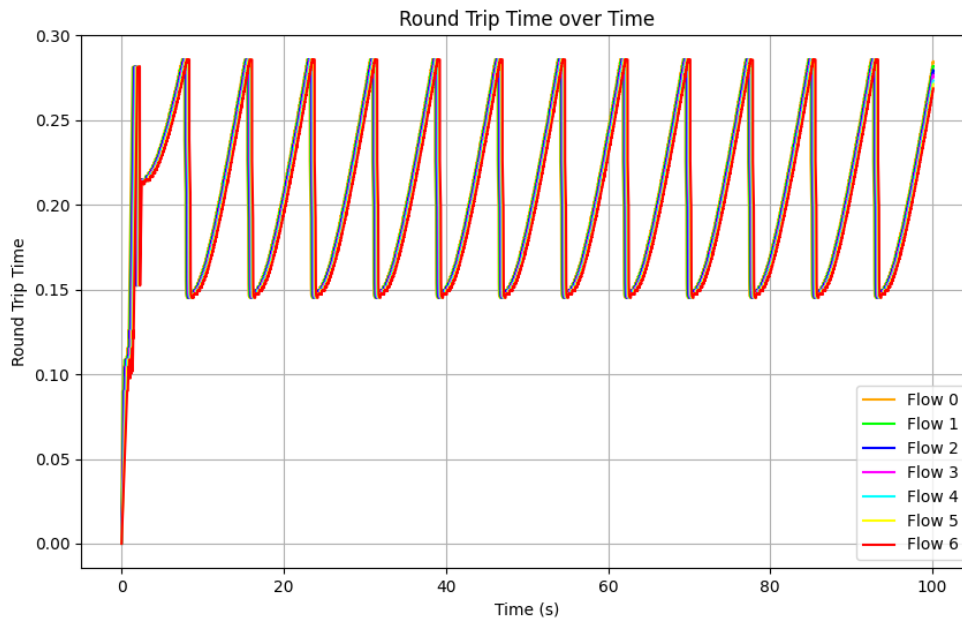
- The growth of alpha is slower compared to the original formula due to the use of the 1.5 power. This results in more gradual CWND growth over time than the Htcp variant.

Slow Start Threshold(ssthresh): The slow start threshold determines when TCP switches from aggressive exponential growth (slow start phase) to more cautious linear growth (congestion avoidance phase). It's like a speed limit that TCP sets for itself based on previous congestion experiences. When cwnd reaches ssthresh, TCP becomes more conservative in its growth



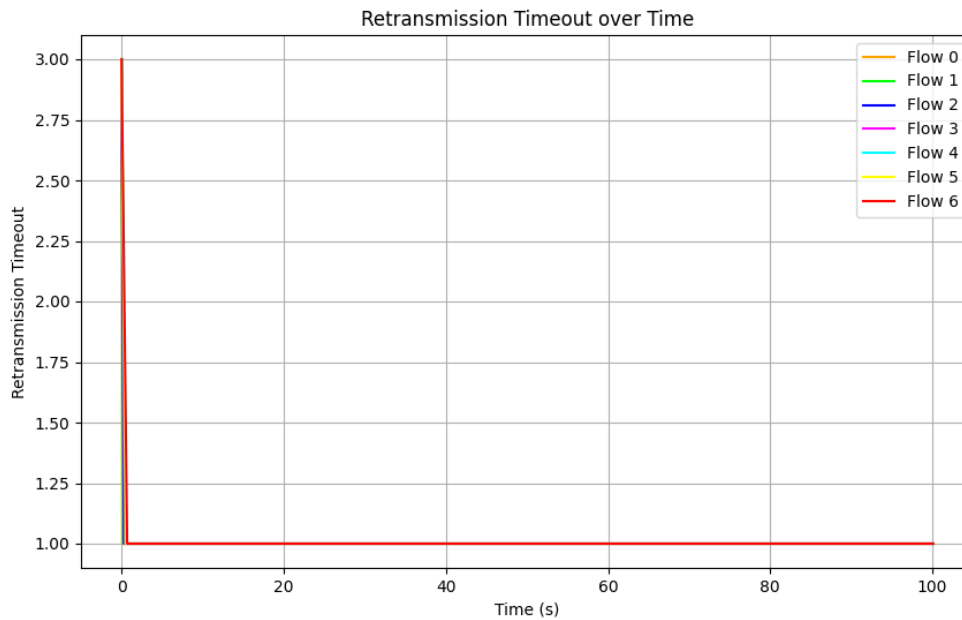
- Exactly same as original Htcp

Round Trip Time (RTT): RTT measures how long it takes for a packet to reach its destination and for an acknowledgment to return.



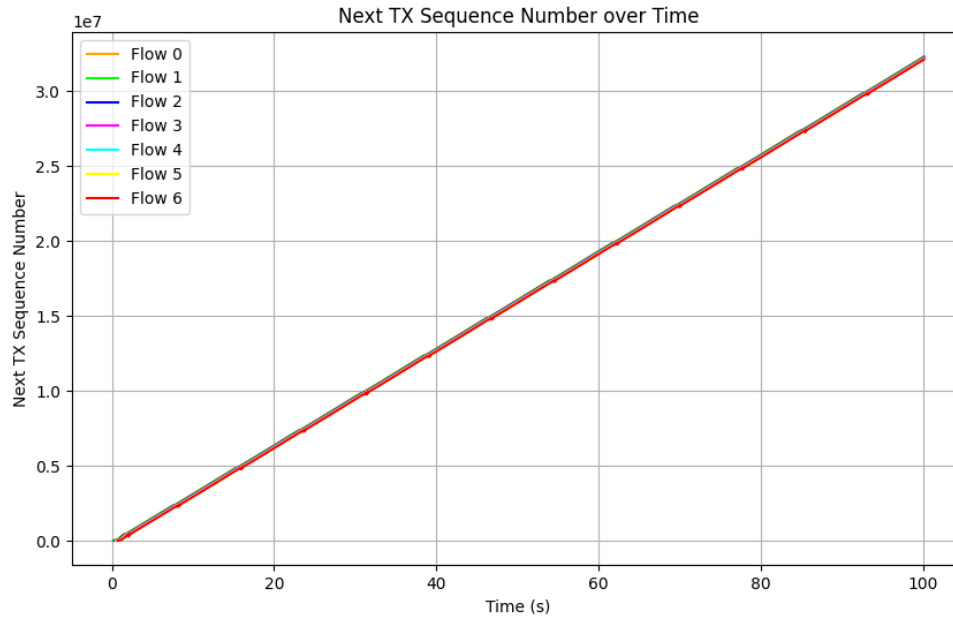
- Less Aggressive growth of RTT.

Retransmission Timeout (RTO): RTO determines how long TCP waits before assuming a packet is lost and needs retransmission. It's like a safety timer - if no acknowledgment arrives within this time, TCP assumes the worst and retransmits. The RTO value is typically calculated based on RTT measurements and their variance.



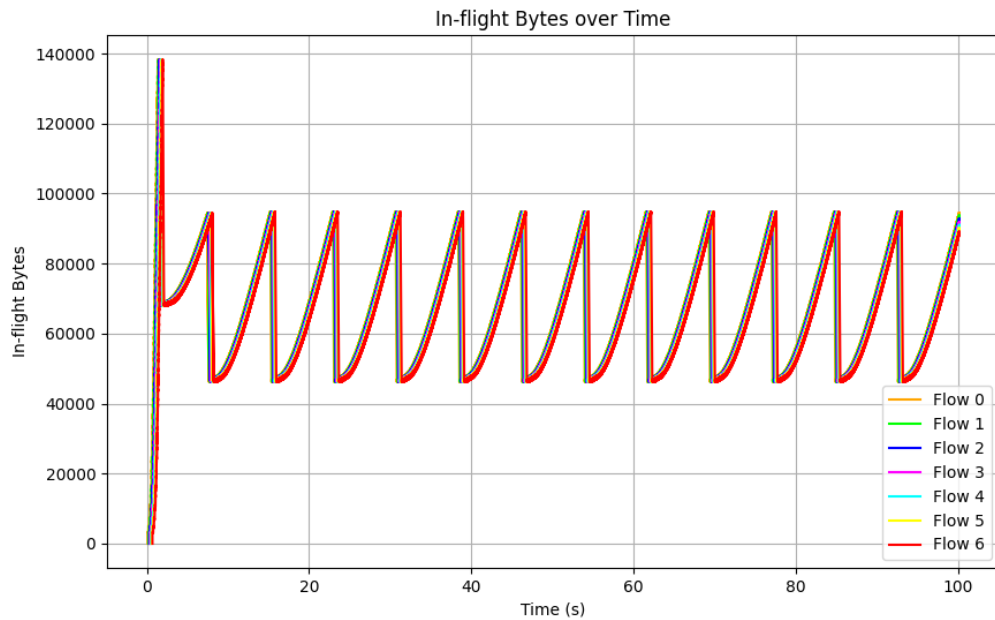
- Exactly same as the original Htcp variant.

Next Transmission Sequence Number (next-tx): This tracks the sequence number of the next packet to be transmitted. It's like a counter that helps ensure packets are sent in order.



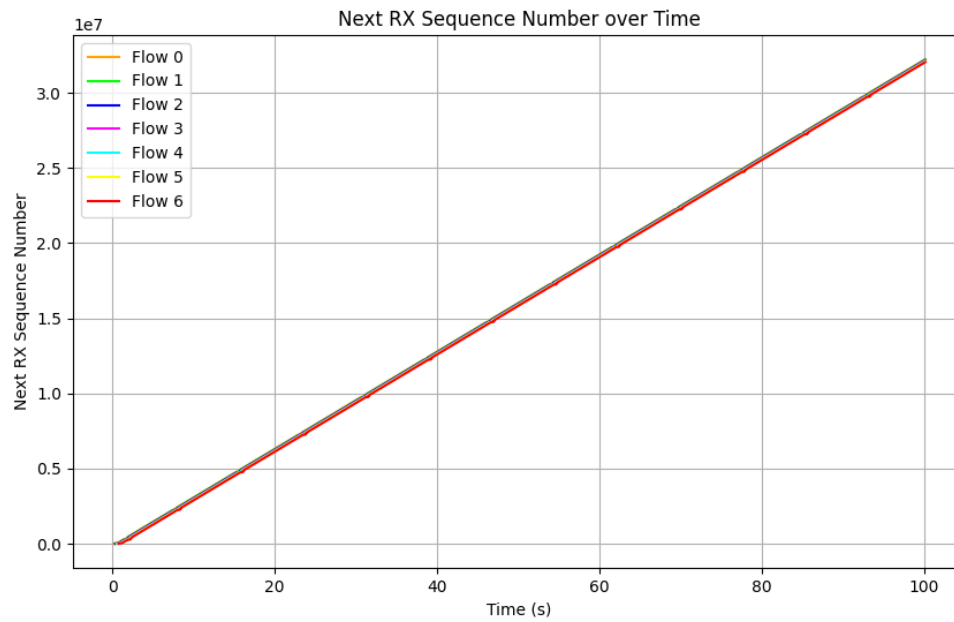
- Exactly same as the original Htcp variant.

In-flight Bytes: This measures how many bytes are currently in transit - sent but not yet acknowledged. It's like tracking packages that are currently in delivery.



- Less Aggressive growth of In-flight bytes.

Next Receive Sequence Number (next RX): This indicates which sequence number the receiver expects next. It helps detect missing or out-of-order packets.



- Exactly same as the original Htcp variant.